

Amazon Sales Data Analysis Project Report



1. Dataset Overview

The dataset used for this project is Amazon Sales Data, which contains transactional-level information related to customer orders. Each row represents an individual order, and the dataset includes attributes such as order date, product category, quantity sold, sales amount, order status, fulfilment type, and shipping details (city, state, country).

This dataset helps analyze:

- Sales performance
- Customer purchasing behavior
- Geographic demand patterns
- Operational efficiency (delivery, fulfilment)

Integrated Development Environment (IDE)

- **Jupyter Notebook (Anaconda)** was used as the IDE. It allows interactive execution of code, visualization, and documentation in a single environment.

Python Libraries Used

- **Pandas:** Data loading, cleaning, manipulation, grouping, and aggregation
- **NumPy:** Numerical analysis such as mean, standard deviation, correlation, missing value detection, and conditional operations
- **Matplotlib:** Basic visualizations like line charts and bar graphs
- **Seaborn:** Advanced and statistical visualizations such as boxplots, count plots, and distribution plots
- **Scikit-learn (sklearn):** Used for advanced analytical concepts such as Isolation Forest (outlier detection)

Key Functions Applied

- Data Cleaning: `isna()`, `fillna()`, `to_numeric()`, `clip()`
- NumPy Analysis: `np.mean()`, `np.std()`, `np.sum()`, `np.where()`, `np.corrcoef()`
- Grouping & Aggregation: `groupby()`, `sum()`, `count()`
- Visualization: `histplot()`, `boxplot()`, `countplot()`, `plot()`

Univariate Analysis:

`data.head()`

This pandas DataFrame method shows the first five rows and all of the accessible columns, allowing for a rapid inspection of the dataset. It enables you to view the types of values recorded in each column, comprehend the data's structure, and confirm that the dataset was loaded correctly. This approach does not alter the actual data; it is only used for previewing.

```
data.tail()
```

Following cleaning, the final values for columns including order details, product information, quantities, prices, and sales-related variables are displayed. The last five records of the cleaned Amazon sales dataset indicate the most recent entries in the dataset order. Verifying that the dataset has been fully loaded, that the cleaning procedures did not cause problems at the end of the data, and that the final records adhere to the same formatting and structure as the rest of the dataset can all be done by viewing `data.tail()`.

```
type(data)
```

It is used to confirm that the pandas DataFrame containing the Amazon sales dataset has been loaded successfully. Because the dataset needs to be in DataFrame format in order for the notebook to proceed with data analysis, cleaning, and visualization, this validation is crucial.

```
data.shape
```

It is used to calculate the size of the Amazon sales dataset, and the result (129175, 24) indicates that there are 24 fields (columns) and 129,175 records (rows) in the dataset. This suggests a sizable dataset with numerous distinct sales entries, each of which is described by 24 distinct parameters pertaining to Amazon sales data, allowing for in-depth research.

```
data.info()
```

It is used to obtain a thorough overview of the Amazon sales dataset, encompassing information on the quantity of rows, columns, data types, and missing values. Together with the data type for each column, the output indicates that the dataset has 24 columns and 129,175 entries. The columns that have missing values can be

identified by comparing the Non-Null Count with the total number of entries. For instance, the columns `courier_status`, `currency`, `amount`, `promotion_ids`, `fulfilled_by`, and `unnamed_cols` have less non-null values, suggesting incomplete data in those fields. This result additionally validates that the dataset contains a variety of data types, including object (categorical/text) columns, booleans, floats, and integers.

```
data.describe(include="all").T
```

It is used to produce a thorough synopsis of the Amazon sales data and its statistical results. The transpose (.T) format organizes the findings such that each dataset column appears as a row for simpler comprehension after the algorithm computes descriptive statistics for every column, including numerical and categorical data. The output reveals that there are 129,175 sales records in the dataset, and that the majority of purchases were shipped and completed by Amazon itself. Values like `shipped`, `amazon`, and `amazon.in` predominate in categorical columns like `status`, `fulfillment`, and `sales_channel`. While the high maximum amount indicates the presence of bulk or large-value orders, numerical columns like `quantity` and `amount` show that the majority of orders are single-item purchases and that the typical order value falls within the mid-range. The majority of orders are shipped within India, according to shipping records, with states like Maharashtra and places like Bengaluru showing up most frequently.

```

# FUNCTION DEFINED TO FIND OUTLIERS

def check_outliers_iqr(df, col):

    Q1 = df[col].quantile(0.25)

    Q3 = df[col].quantile(0.75)

    IQR = Q3 - Q1

    lower = Q1 - 1.5 * IQR

    upper = Q3 + 1.5 * IQR

    outliers = df[(df[col] < lower) | (df[col] > upper)]

    print(f"{col}")

    print("Lower Bound:", lower)

    print("Upper Bound:", upper)

    print("Outlier Count:", outliers.shape[0])

    print("-" * 40)

# USED THE DEFINED FUNCTION

for col in ["amount", "qty"]:

    check_outliers_iqr(data, col)

```

The function `check_outliers_iqr` is defined to **identify outliers in numerical columns of the Amazon sales dataset using the Interquartile Range (IQR) method**. The output, when applied to the `amount` and `qty` columns, indicates which records are outside the normal range and how widely values usually spread. The result shows 3,636 high-value transactions, indicating the presence of bulk or extremely expensive orders. The lower bound for the `amount` column is -59.5 , while the upper bound is 1296.5 . Any order value exceeding ₹1296.5 is considered an outlier. The 13,879 outliers indicate orders with quantities greater than one, emphasizing bulk purchases; the bottom and upper bounds for the `quantity` column are both 1.0, indicating that the majority of orders contain exactly one item. All things

considered, the code and output work together to identify odd sales trends that might require more care or independent dataset study.

```
# HANDLING THE OUTLIER BY CAPPING METHOD

def cap_outliers_iqr(df, col):

    Q1 = df[col].quantile(0.25)

    Q3 = df[col].quantile(0.75)

    IQR = Q3 - Q1

    lower = Q1 - 1.5 * IQR

    upper = Q3 + 1.5 * IQR

    df[col] = np.where(df[col] < lower, lower,

                      np.where(df[col] > upper, upper, df[col]))

# USING THE ABOVE FUNCTION

for col in ["qty", "amount"]:

    cap_outliers_iqr(data, col)

# AGAIN CHECKING THE OUTLIERS REMOVED OR NOT

for col in ["amount", "qty"]:

    check_outliers_iqr(data, col)
```

To deal with outliers in the Amazon sales dataset, the function `cap_outliers_iqr` applies the capping strategy based on the IQR rule. The code limits extreme values by substituting the lower bound for any value below it and the upper bound for any value above it, rather than deleting data. The quantity and amount columns, which previously displayed notable outliers, are subjected to this method. Extreme values have been successfully capped inside the permitted range, as evidenced by the output showing an outlier count of 0 for both columns when the outlier check is performed again. This guarantees that all sales records are kept in the dataset and keeps abnormally high volumes or amounts from distorting subsequent analysis.

```
# TO CHECK THE SUM OF NULL  
VALUES IN EACH COLUMN
```

```
data.isnull().sum()
```

It is applied to each column of the Amazon sales dataset in order to detect and count missing values. The output demonstrates that there are no missing values in a large number of essential variables, including order_id, status, quantity, and category, showing good data completeness for important sales information. However, a number of columns, such as promotion_ids, fulfilled_by, and unnamed_cols, have conspicuous null values, indicating that fulfillment and promotional data are unavailable for a significant percentage of orders. There are also some missing entries in columns like courier_status, currency, amount, ship_city, and ship_state, which show incomplete shipping and pricing information.

```
# TO DROP AND HANDLE THE NULL VALUES
```

```
data.drop(columns=['promotion_ids', 'unnamed_cols', 'fulfilled_by'], inplace=True)
```

```
# TO REPLACE THE OBJECT TYPE VALUES WITH UNKNOWN
```

```
object_missing_values = [ 'courier_status', 'currency', 'ship_city', 'ship_state',  
'ship_postal_code', 'ship_country' ]
```

```
data[object_missing_values] = data[object_missing_values].fillna('Unknown')
```

```
# REPLACE THE NUMERIC VALUES OF COLUMN AMOUNT WITH MEDIAN
```

```
data.fillna({'amount': data['amount'].median()}, inplace=True)
```

Depending on the type and significance of each column, this method handles missing values in the Amazon sales dataset in an organized manner. In order to reduce noise in the dataset, columns like promotion_ids, unnamed_cols, and fulfilled_by are completely removed because they have a large amount of null values and don't have any significant analytical value. Missing values are substituted with "Unknown" for categorical and location-related fields such as courier_status, currency, ship_city, ship_state, ship_postal_code, and ship_country in order to

maintain all sales records while conspicuously indicating information that is not accessible. Last but not least, the median order amount a reliable option that prevents distortion from extreme values is used to fill in the missing numbers in the numeric amount column. By eliminating weak columns, normalizing missing categorical data, and stabilizing numerical values, these procedures together clean the dataset and prepare it for trustworthy analysis.

```
# CHECK AGAIN THE MISSING VALUES HANDLED CORRECTLY OR NOT  
data.isnull().sum()
```

`data.isnull().sum()` is used again to **verify whether missing values have been handled correctly in the Amazon sales dataset**. The output shows **0 null values for all remaining columns**, confirming that the previously applied steps dropping highly incomplete columns, replacing missing categorical values with *“Unknown”*, and filling missing numeric values in amount with the median were successful. This result indicates that the dataset is now complete and consistent, making it ready for further analysis and modeling without issues related to missing data.

```
# TO FIND THE DUPLICATE VALUES  
data.duplicated().sum()  
  
# TO DROP THE DUPLICATE VALUES  
data.drop_duplicates()
```

The return number 200 shows that there are 200 duplicate sales entries in the Amazon sales dataset, which is found using `data.duplicated().sum()`. If left unchecked, these duplicates, which might arise from data collecting or merging procedures, could skew analytical results. To make sure that every row in the dataset represents a distinct sales entry, these duplicate records are eliminated using the `data.drop_duplicates()` function. All further analysis is more accurate and reliable after this cleaning phase.


```
# SORTED THE DATA BY PARSED DATE  
data.sort_values(by='parsed_date')
```

Using the `parsed_date` column as a guide, `sort_values(by='parsed_date')` is used to put the Amazon sales dataset in chronological order. By arranging all sales records from earlier to later dates, this sorting facilitates the analysis of time-based patterns in the dataset, including daily sales trends, order flow across time, and seasonal behavior.

```
# TO CALCULATE THE SUM OF AMOUNT COLUMN USING SUM  
FUNCTION OF NUMPY LIBRARY  
np.sum(data['amount'])
```

The overall sales value from the Amazon sales dataset is determined by adding up all of the values in the amount column using `np.sum(data['amount'])`. After accounting for outliers and missing numbers, the total revenue from all recorded orders is shown in the output, 82925437.93. This outcome is helpful for comprehending the overall business volume recorded in the Amazon sales data since it shows the total sales amount for the full dataset.

```
# TO CALCULATE THE AVERAGE SALE USING BELOW FUNCTION OF NUMPY  
np.mean(data['amount'])
```

The average order value in the Amazon sales dataset is determined by `np.mean(data['amount'])`; the result 641.96 shows that, on average, each order brings in about ₹642 in sales. This figure gives insight into the general purchasing patterns of customers and represents the average purchase amount following data cleaning, which includes handling missing values and outliers.

```
# TO CALCULATE THE MEDIAN OF AMOUNT COLUMN
```

```
np.median(data['amount'])
```

When the middle value of sales amounts in the Amazon sales dataset is determined using `np.median(data['amount'])`, the output 605.0 shows that half of the orders have sales amounts less ₹605 and half have sales quantities over ₹605. The median is a more reliable indicator of central tendency for the dataset since it displays the average order value unaffected by expensive purchases.

```
# TO CALCULATE THE PERCENTILES OF AMOUNT COLUMN
```

```
np.percentile(data['amount'], [25, 50, 75, 90, 95])
```

To determine how sales amounts are distributed across various levels in the Amazon sales dataset, use `np.percentile(data['amount'], [25, 50, 75, 90, 95])`. 25% of orders are below ₹459, 50% (the median) are below ₹605, and 75% are below ₹771, according to the output, suggesting that the majority of purchases fall under this range. Only a small percentage of orders have larger sales values, as indicated by higher percentiles, which reveal that 90% of orders are below ₹1033 and 95% are below ₹1168. This distinguishes higher-value transactions from regular purchases and aids in the identification of spending trends.

```
# TO CHECK THE LOW AND HIGH VALUES FROM THE AVERAGE SALES AMOUNT
```

```
data['sales_level'] = np.where(  
    data['amount'] > np.mean(data['amount']),  
    'High Sales',  
    'Low Sales'  
)  
print(data[['amount', 'sales_level']])
```

Based on the average sales amount, this code categorizes every order in the Amazon sales dataset as either a high or low sales transaction. A new column named `sales_level` is established using the mean of the amount column (about ₹642), and orders with amounts higher than the average are labeled "High Sales," while orders with amounts equal to or lower than the average are branded "Low Sales." For all 129,175 records, the output displays the amount along with its matching `sales_level`. This demonstrates that most orders are in the Low Sales category, which is consistent with the usual buying patterns seen previously in the dataset.

```
# ENCODING Courier status

from sklearn.preprocessing import LabelEncoder

# CREATE A LABELENCODER OBJECT

label_encoder=LabelEncoder()

# FIT AND TRANSFORM the COLUMN

data["Courier status encoded"] =
label_encoder.fit_transform(data["status"])

print(data[["status", "Courier status encoded"]])
```

In order to use it in analysis or machine learning models, this function transforms the Amazon sales dataset's category status column into numerical form. Each order status (shipped, cancelled, etc.) is given a distinct numeric value by the `LabelEncoder`, and these encoded values are stored in the new column `Courier status encoded`. In order to verify that every status category has been reliably converted into a number while maintaining the differentiation between various order states, the output displays the original status along with its numeric label.

```
# ENCODING fulfillment

from sklearn.preprocessing import LabelEncoder

# CREATE A LABELENCODER OBJECT

label_encoder=LabelEncoder()

# FIT AND TRANSFORM the COLUMN

data["fulfillment encoded"] =
label_encoder.fit_transform(data["fulfillment"])

print(data[["fulfillment", "fulfillment encoded"]])
```

The Amazon sales dataset's categorical fulfillment column is transformed into numerical values using this code. Every fulfillment type like merchant or Amazon is given a distinct numerical designation and kept in the newly created fulfillment encoded column. The output confirms that the fulfillment method for each order has been successfully converted into a numeric format appropriate for additional analysis.

```
# ENCODING SHIP SERVICE LEVEL

from sklearn.preprocessing import LabelEncoder

# CREATE A LABELENCODER OBJECT

label_encoder=LabelEncoder()

# FIT AND TRANSFORM the COLUMN

data["ship_service_level encoded"] =
label_encoder.fit_transform(data["ship_service_level"])

print(data[["ship_service_level", "ship_service_level encoded"]])
```

The Amazon sales dataset's categorical ship_service_level column is converted to numeric values using this code. The new column ship_service_level encoded contains a unique number label for each shipping service level (e.g., expedited or standard). The output verifies that the shipping speed data has been correctly

converted into a numerical format for additional analysis or modeling by displaying the original shipping service level together with its encoded value.

```
# ENCODING SHIP STATE

from sklearn.preprocessing import LabelEncoder

# CREATE A LABELENCODER OBJECT

label_encoder=LabelEncoder()

# FIT AND TRANSFORM the COLUMN

data["ship_state encoded"] =
label_encoder.fit_transform(data["ship_state"])

print(data[["ship_state","ship_state encoded"]])
```

The Amazon sales dataset's ship_state column, which is categorical, is transformed into numerical form using this code. A distinct numerical designation is given to each shipping state and is kept in the newly created ship_state encoded column. In order to verify that geographic shipping data has been converted into a format appropriate for analysis or machine learning operations, the output shows the original state names together with their encoded values.

```
# ENCODING SHIP CITY

from sklearn.preprocessing import LabelEncoder

# CREATE A LABELENCODER OBJECT

label_encoder=LabelEncoder()

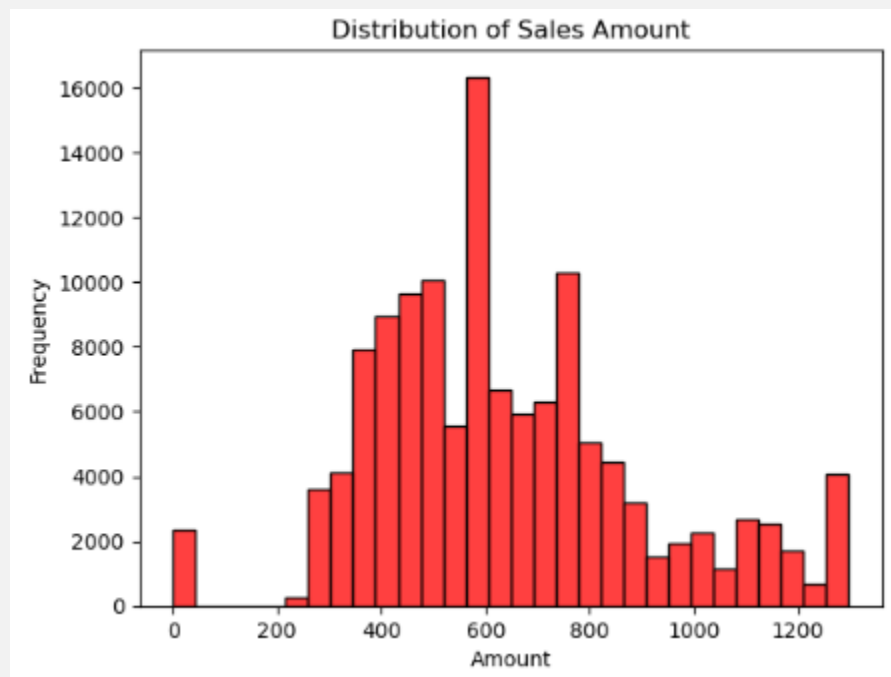
# FIT AND TRANSFORM the COLUMN

data["ship_city encoded"] =
label_encoder.fit_transform(data["ship_city"])

print(data[["ship_city","ship_city encoded"]])
```

The Amazon sales dataset's categorical ship_city column is transformed into numerical values using this code. A distinct numerical label is given to each shipping city and is kept in the newly created ship_city encoded column. The output confirms that city-level shipping data has been converted into a numeric format that is simple to utilize for analysis and modeling by displaying the original city names together with their encoded values.

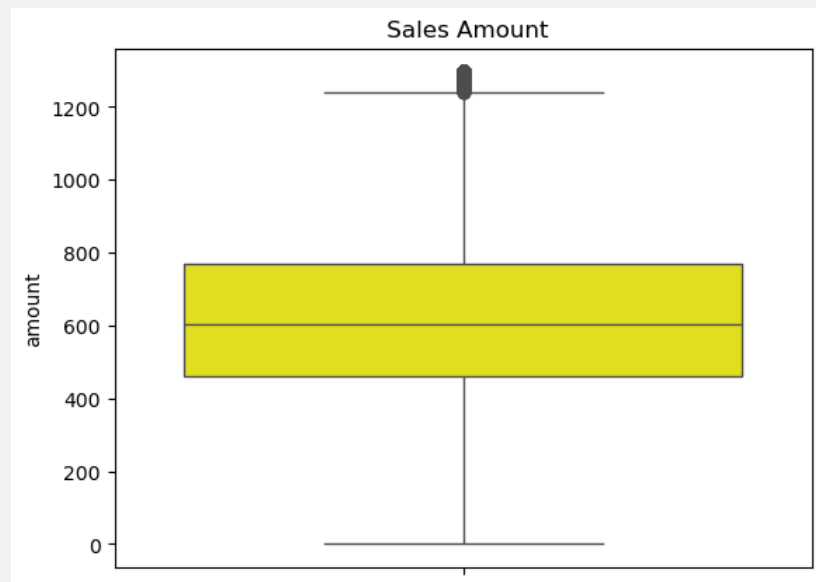
Visualization 1: Distribution of sales amount



The produced histogram illustrates how frequently various sales values occur. This code visualizes the distribution of order amounts in the Amazon sales dataset. With a distinct peak at ₹600, the figure shows that a significant portion of orders are centered in the ₹400–₹800 range, which is consistent with the mean and median sales values that were previously determined. A right-skewed distribution results from comparatively fewer orders at very low amounts and gradually less as the

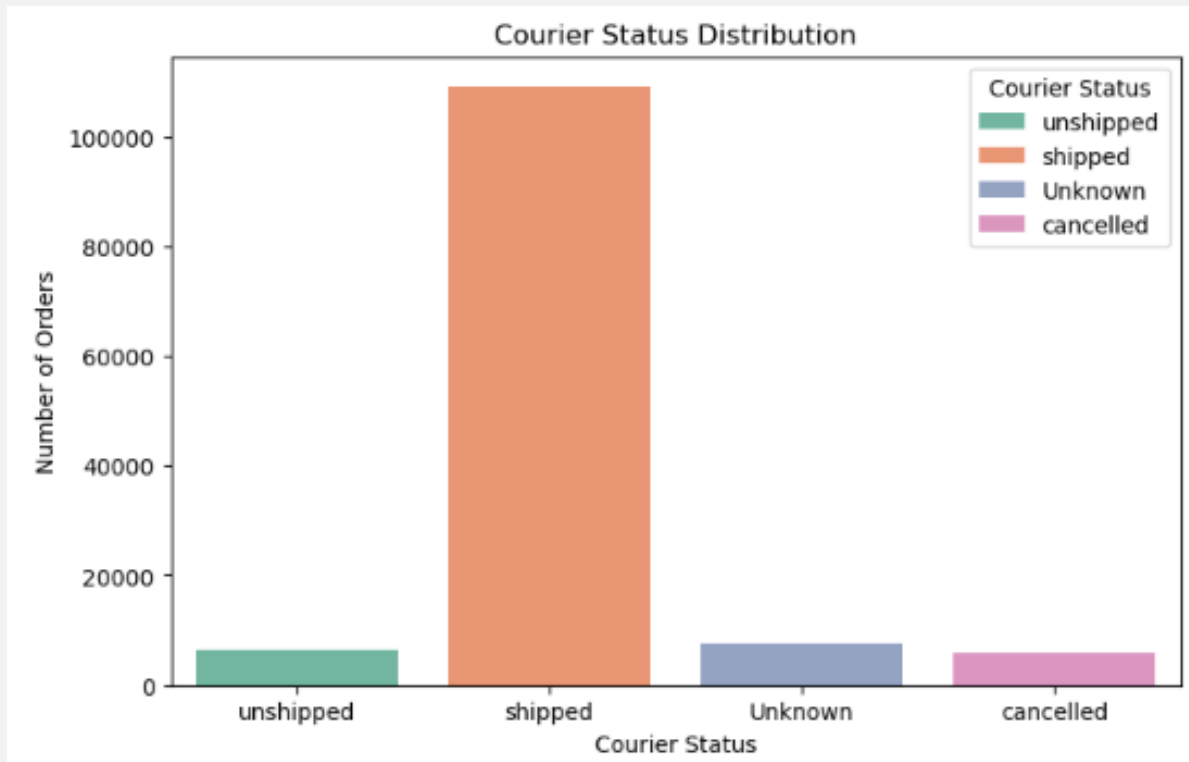
amount rises. This demonstrates that while high-value transactions do occur in the dataset, they are far less frequent than low- to mid-value orders placed by the majority of clients.

Visualization 2: Detection of Outliers in sales amount



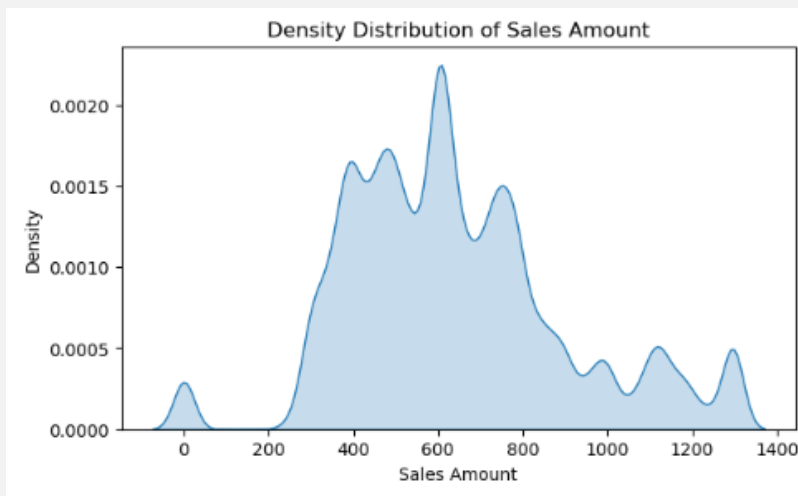
Using a boxplot, this algorithm finds outliers in the sales amount of the Amazon sales dataset. The boxplot displays the median, interquartile range, and extreme values to visually summarize the amount column's distribution. The output shows that the median sales amount is approximately ₹600, and the majority of orders fall inside the box's center range. A tiny number of orders with abnormally high sales quantities are represented by the dots plotted above the upper whisker, which are high-value outliers. Outliers that were previously found and managed with the IQR-based capping method are confirmed to exist in this visualization.

Visualization 3: Distribution of Courier status



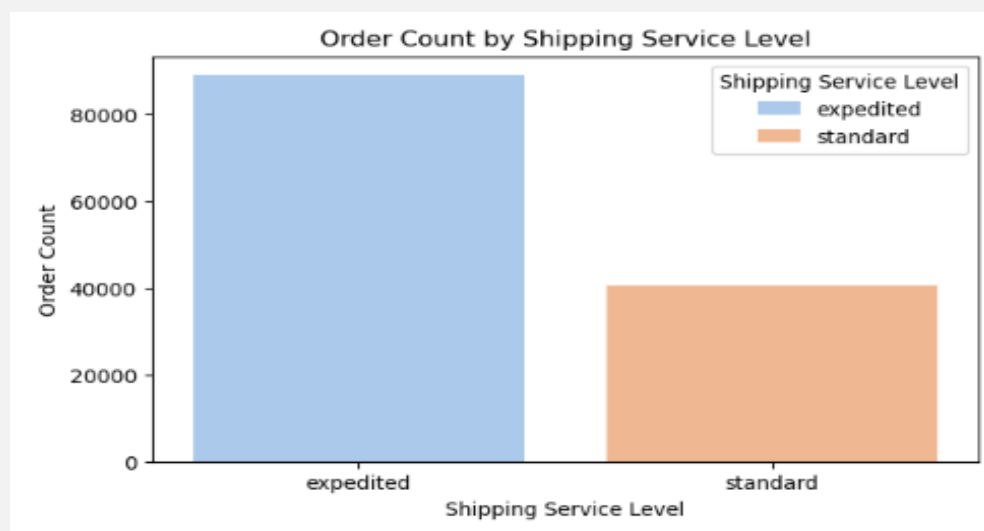
This code uses a count plot to display the courier status distribution in the Amazon sales dataset. The data clearly reveals that the majority of orders are tagged as "shipped," indicating successful order fulfillment. The chart displays the number of orders under each courier status category. The "Unknown" category relates to orders whose courier information was absent and subsequently included during data cleaning, while the smaller figures for "unshipped" and "cancelled" indicate orders that were either not shipped or canceled. Overall, this visualization shows that most sales proceeded to successful dispatch and aids in evaluating order fulfillment performance.

Visualization 4: Distribution of sales amount



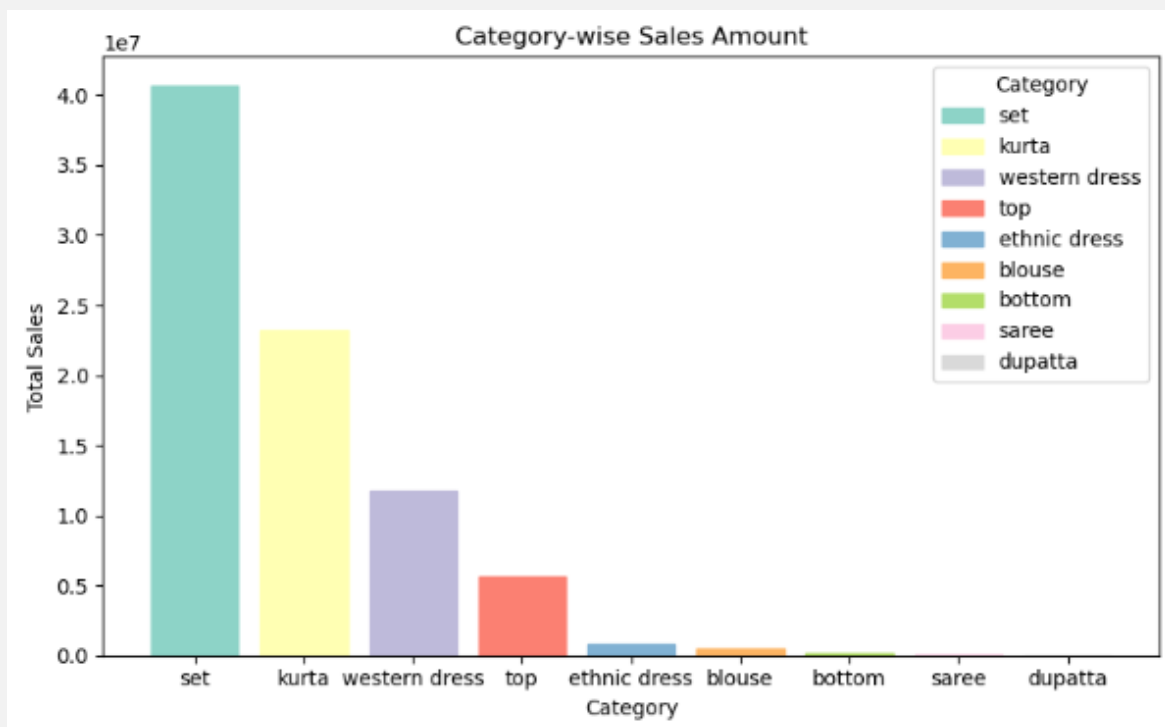
This visualization illustrates how sales values are distributed across various price points by plotting the density distribution of sales amounts in the Amazon sales dataset. The KDE curve shows that, in line with the previously determined mean and median sales values, the largest density of orders is concentrated between ₹500 and ₹700, with a distinct peak close to ₹600. Very low sales amounts only happen seldom, whereas high-value orders are less common, as seen by the curve's steady tapering toward higher values. The smooth depiction of sales behavior in this visualization demonstrates that low- to mid-range purchases predominate in the dataset.

Visualization 5: Number of orders by shipping service level



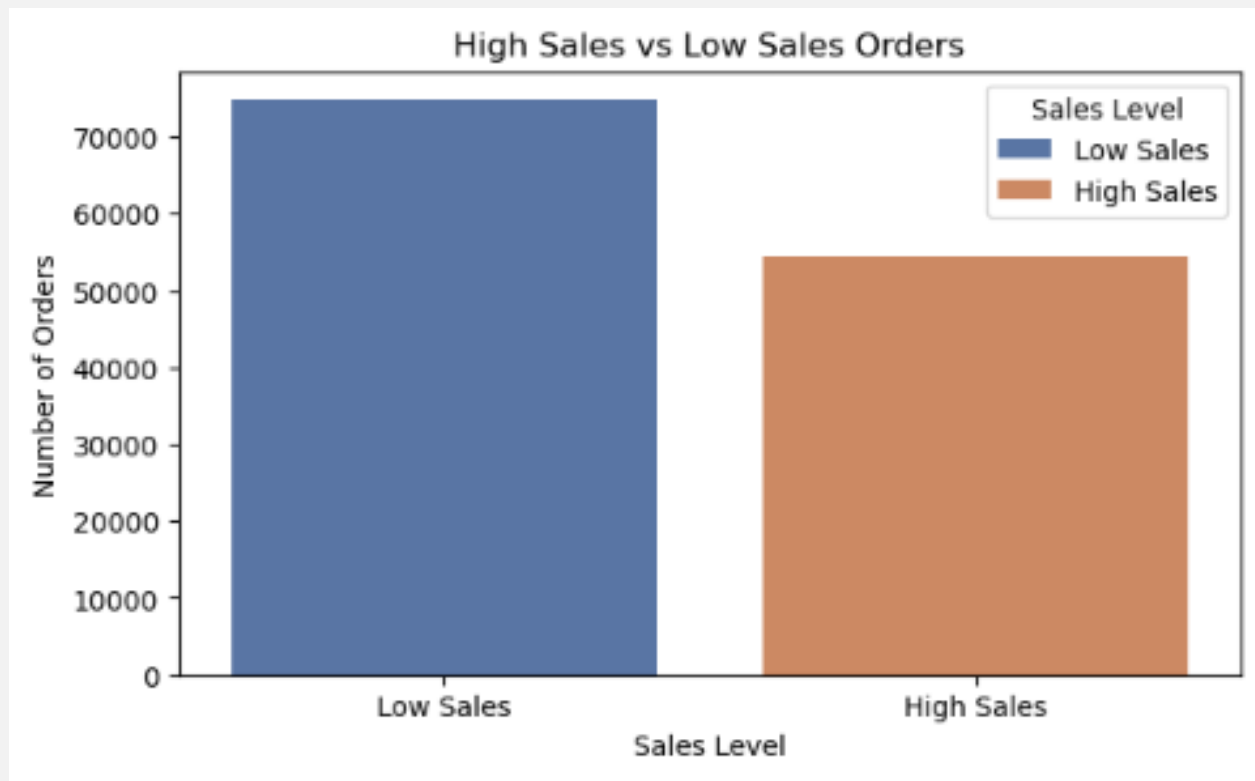
The number of orders by shipping service level in the Amazon sales dataset is displayed using this code. The count plot demonstrates that, in comparison to the normal option, a noticeably higher percentage of orders choose the accelerated shipping service. This suggests that consumers in the sample typically favor quicker delivery, either as a result of urgency or the type of goods being bought. The graphic shows expedited shipping as the most popular service level and aids in comparing customer shipping choices.

Visualization 6: Sales amount according to category



The total sales amount for each product category in the Amazon sales dataset is computed and shown using this code. To identify the best-performing categories, the data is first grouped by category, and then the sales totals are added together and arranged in descending order. While categories like saree and dupatta contribute relatively low sales, the "set" category yields the most overall sales, followed by kurta and western dress, according to the bar chart output. This graphic helps determine which product categories generate the highest revenue in the dataset and compares category performance in an easy-to-understand manner.

Visualization 7: Number of High and low sales orders



Based on the average sales amount, this Visualization shows the number of orders in the Amazon sales dataset that are categorized as High Sales and Low Sales. The count plot indicates that the majority of purchases are below the average order value, with Low Sales orders being more common than High Sales orders. Despite being less frequent, high sales orders signify transactions with values above average and make a substantial contribution to total revenue. This graphic supports the preceding finding that customer purchases are often low to mid-value and aids in comparing order volume throughout sales levels.

Bivariate Analysis:

```
# To check the sales level  
data.groupby('sales_level').size()
```

group by data ('sales_level').The Amazon sales dataset's size() function counts the quantity of orders in each sales category. A greater percentage of orders fall below the average sales value, as evidenced by the output, which displays 54,419 High Sales orders and 74,756 Low Sales orders. This supports the previous visual study that there are more low-value orders and fewer high-value orders.

```
# TO CHECK TOTAL SALES BY STATE  
data.groupby('ship_state')['amount'].sum().sort_values(ascending=False)
```

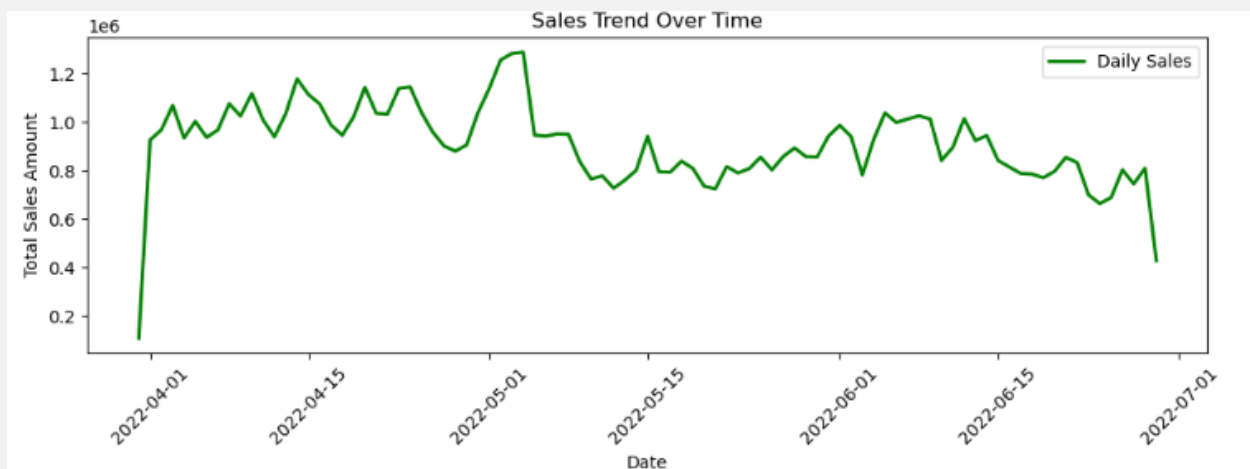
To highlight the best-performing areas, this code computes the total sales amount for each shipping state in the Amazon sales dataset and arranges the results in descending order. According to the results, Telangana and Karnataka contribute the most to total revenue, while Maharashtra earns the greatest total sales. While smaller regions and union territories generate relatively lower sales quantities, states such as Uttar Pradesh, Tamil Nadu, and Delhi also have substantial sales performance. Orders with missing state information that were later filled up during data cleaning are indicated by the existence of a "Unknown" category. All things considered, this study aids in identifying the major geographic regions that propel sales.

```
# TO CALCULATE AVERAGE SALES BY FULFILLMENT TYPE  
data.groupby('fulfillment')['amount'].mean()
```

The average sales amount for each fulfillment type in the Amazon sales dataset is determined by this code. According to the output, the average sales value of orders

completed by Amazon is ₹642.46, whereas the average sales value of orders fulfilled by merchants is ₹640.81. Given that consumers often spend almost the same amount whether their orders are fulfilled by Amazon or a merchant, this suggests that the fulfillment method has little effect on the average order value.

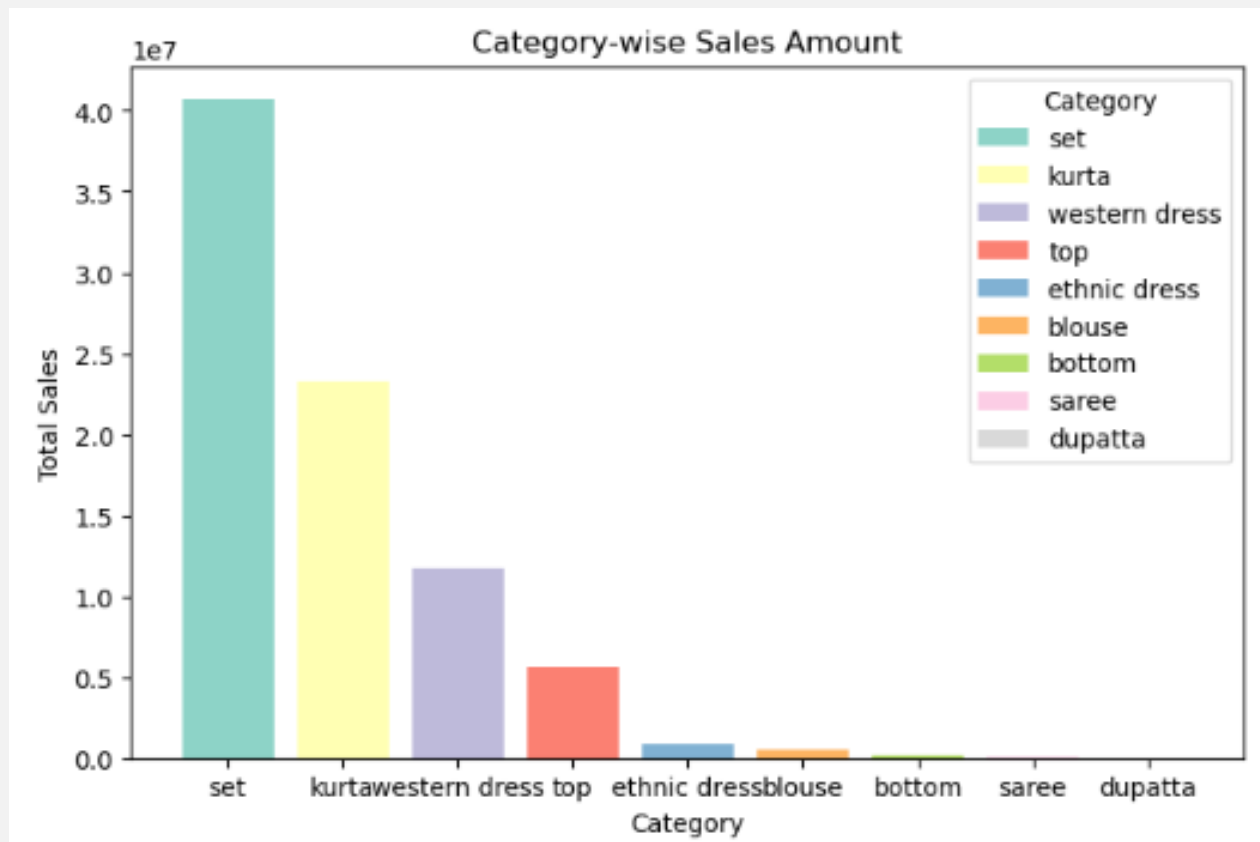
Visualization 8: To check the sales trend over time



This shows the Amazon sales dataset's daily sales trend over time. To guarantee precise time-based grouping, the `parsed_date` column is first transformed into the appropriate datetime format. The total sales amount (amount) is then calculated for each day and arranged chronologically after the data has been grouped by date. With dates on the x-axis and the total sales amount on the y-axis, the resulting line plot illustrates how total daily sales vary over time.

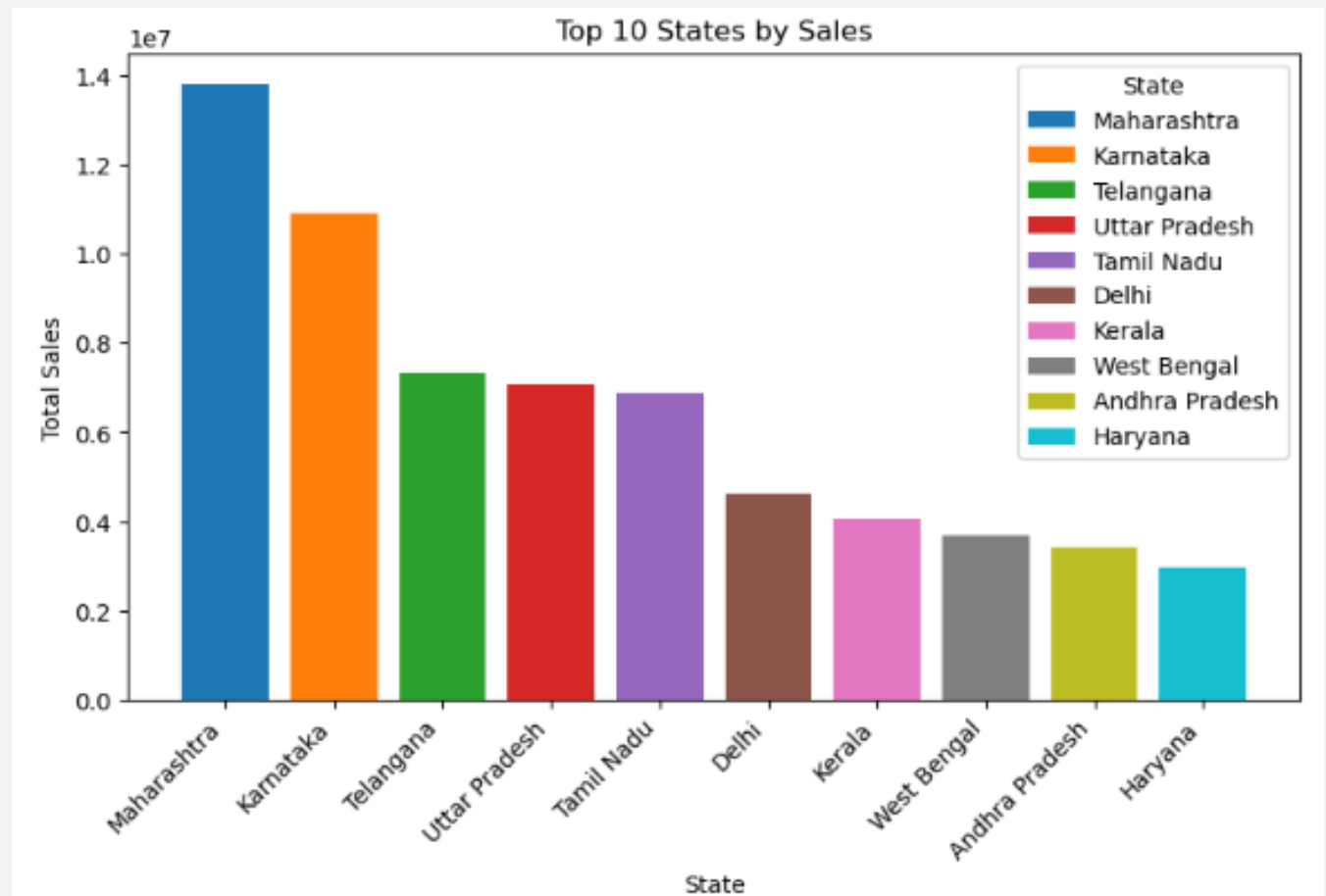
The output graph shows a period of significant sales activity, with daily sales beginning at a lower level in early April, rising swiftly, and remaining quite strong through mid-April and early May. Following this peak, sales exhibit a slow fall with discernible oscillations through June, indicating a steady shift in client desire. All things considered, the visualization helps identify peak and low sales periods in the dataset by highlighting sales seasonality and patterns.

Visualization9: Sales according to category of product



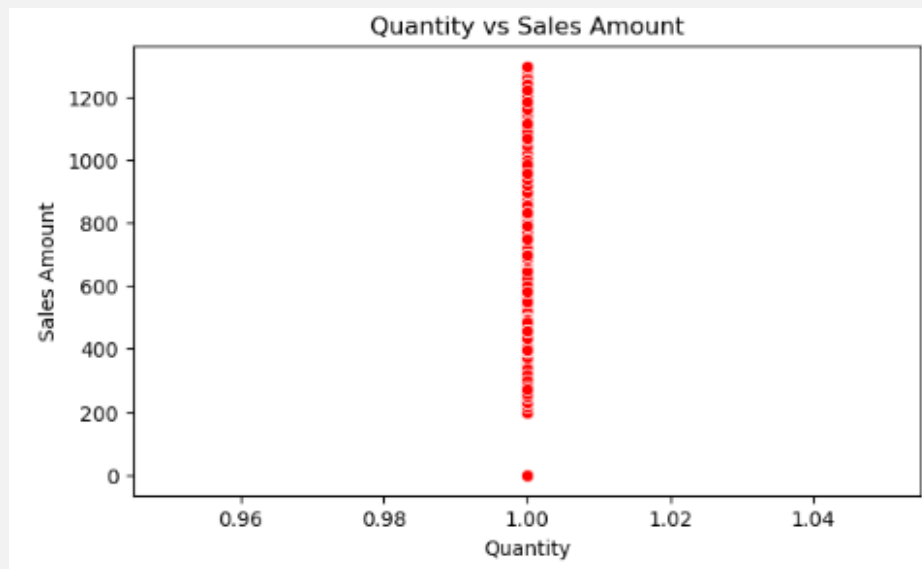
The entire sales amount for each product category is displayed in the bar chart. According to the data, the "Set" category has the most sales, followed by "Kurta" and "Western Dress," suggesting that these categories are highly preferred by customers. Conversely, categories including bottom, dupatta, and saree yield comparatively fewer sales, indicating either decreased purchasing frequency or limited demand. Overall, the picture shows that sales are not evenly distributed across categories, with a small number of categories generating the majority of revenue.

Visualization 10 : Sales in top 10 states



The top ten states by total sales amount are shown in the bar chart. According to the findings, Telangana and Karnataka have the biggest sales contributions, followed by Maharashtra, suggesting that these states have substantial regional demand. While Delhi, Kerala, West Bengal, Andhra Pradesh, and Haryana record somewhat lower sales among the top 10, states like Uttar Pradesh and Tamil Nadu also exhibit noteworthy sales performance. All things considered, the graphic emphasizes regional differences in sales, with a greater concentration of income coming from a small number of important states.

Visualization 11: Quantity sold and sales amount



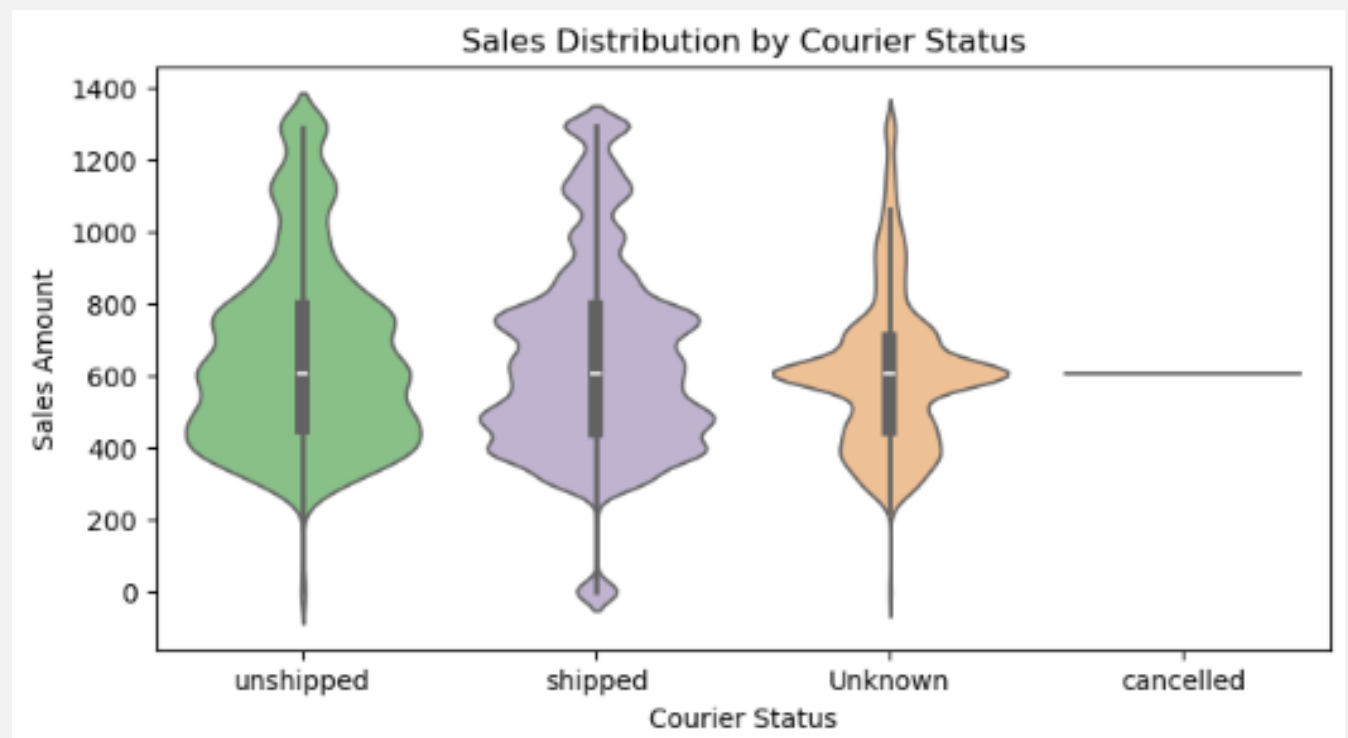
The bulk of orders are single-item purchases, as the graphic demonstrates, with the majority of data points grouped around a quantity value of one. The sales amount fluctuates greatly even while the quantity stays the same, indicating that pricing variations among products rather than the number bought are what cause the fluctuation in sales value. This suggests that when calculating total sales, product pricing has a greater impact than order size.

Visualization 12: Sales distribution according to fulfillment type



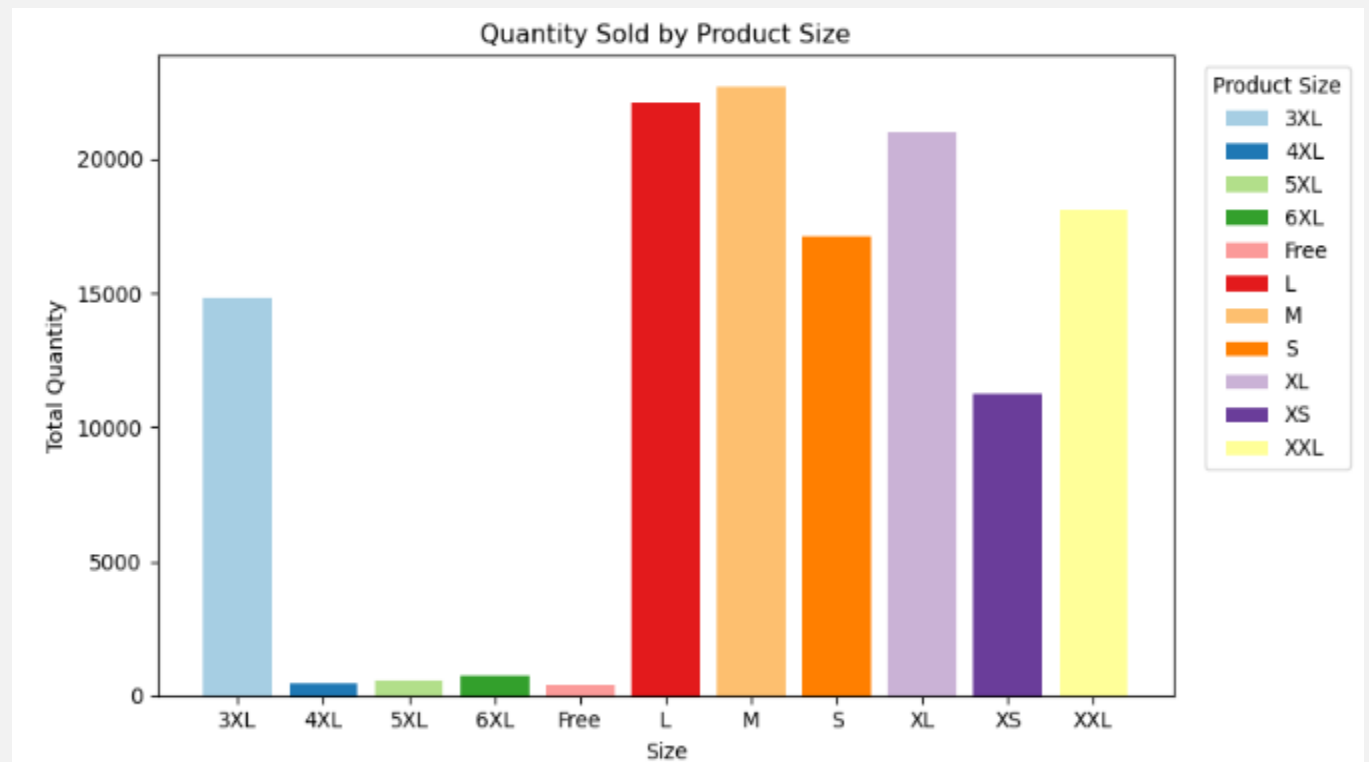
The distribution of sales amounts across various fulfillment options is compared using a box plot. The findings indicate equal performance in terms of sales amount, with both merchant-fulfilled and Amazon-fulfilled orders having a similar median sales value and total dispersion. The existence of multiple high-value outliers in both fulfillment categories, however, points to sporadic high-value transactions. Since both channels show comparable sales distributions, the visualization suggests that the delivery method has little effect on the total sales amount.

Visualization 13: Sales distribution according to Courier status



The distribution of sales quantities across various courier statuses is depicted by the violin plot. Wider distributions and more sales amount variability are seen in orders with unshipped and shipped statuses, suggesting a wide range of order values. Whereas the cancelled status indicates little or constant sales amounts as a result of order termination, the unknown courier status displays a more concentrated distribution around mid-range sales values. According to this data, sales revenue from active delivery stages is higher than that from cancelled orders.

Visualization 14: Quantity sold by product size



The total quantity sold across various product sizes is depicted in the bar chart. Higher customer demand for standard and often worn sizes is reflected in the results, which show that medium (M) and large (L) sizes have the biggest sales volume, followed by extra large (XL) and double extra large (XXL). On the other hand, free size and extreme sizes like 3XL, 4XL, 5XL, and 6XL exhibit somewhat smaller sales numbers. All things considered, the graphic indicates that the majority of consumer purchases center on mid-range sizes, emphasizing how crucial it is to keep enough inventory of these sizes to satisfy demand.

Multivariate Analysis:

```
# TO check State-wise Sales by Sales Level  
data.groupby(['ship_state', 'sales_level'])['amount'].sum()
```

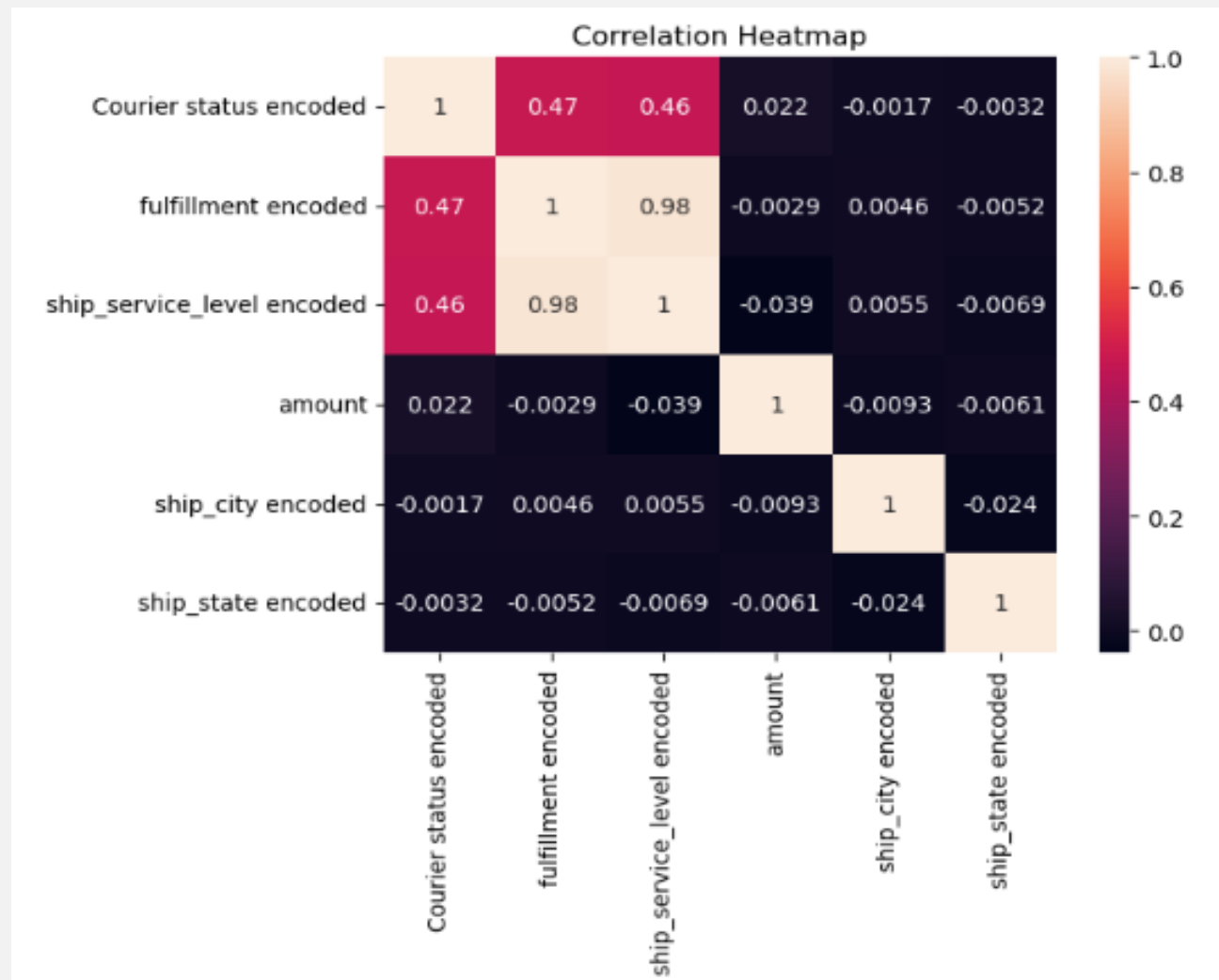
The grouped analysis looks at the distribution of sales by state for both high and low sales levels. The findings show that the total sales amount under the High Sales category is higher than that under the Low Sales category in the majority of states, indicating a higher revenue contribution from high-performing transactions. Significant sales figures in both categories are displayed by major states like Andhra Pradesh, Uttar Pradesh, and West Bengal, underscoring their overall market importance. This analysis offers insights into areas with consistently high revenue creation and helps discover how sales performance differs among states.

```
# CALCULATE CORRELATION MATRIX  
correlation = data[  
    ['Courier status encoded',  
     'fulfillment encoded',  
     'ship_service_level encoded',  
     'amount',  
     'ship_city encoded',  
     'ship_state encoded']  
>].corr()  
  
# DISPLAY THE CORRELATION VALUES  
correlation
```

	Courier status encoded	fulfillment encoded	ship_service_level encoded	amount	ship_city encoded	ship_state encoded
Courier status encoded	1.000000	0.469547	0.463650	0.022463	-0.001666	-0.003165
fulfillment encoded	0.469547	1.000000	0.980479	-0.002922	0.004562	-0.005202
ship_service_level encoded	0.463650	0.980479	1.000000	-0.038630	0.005480	-0.006899
amount	0.022463	-0.002922	-0.038630	1.000000	-0.009312	-0.006109
ship_city encoded	-0.001666	0.004562	0.005480	-0.009312	1.000000	-0.023643
ship_state encoded	-0.003165	-0.005202	-0.006899	-0.006109	-0.023643	1.000000

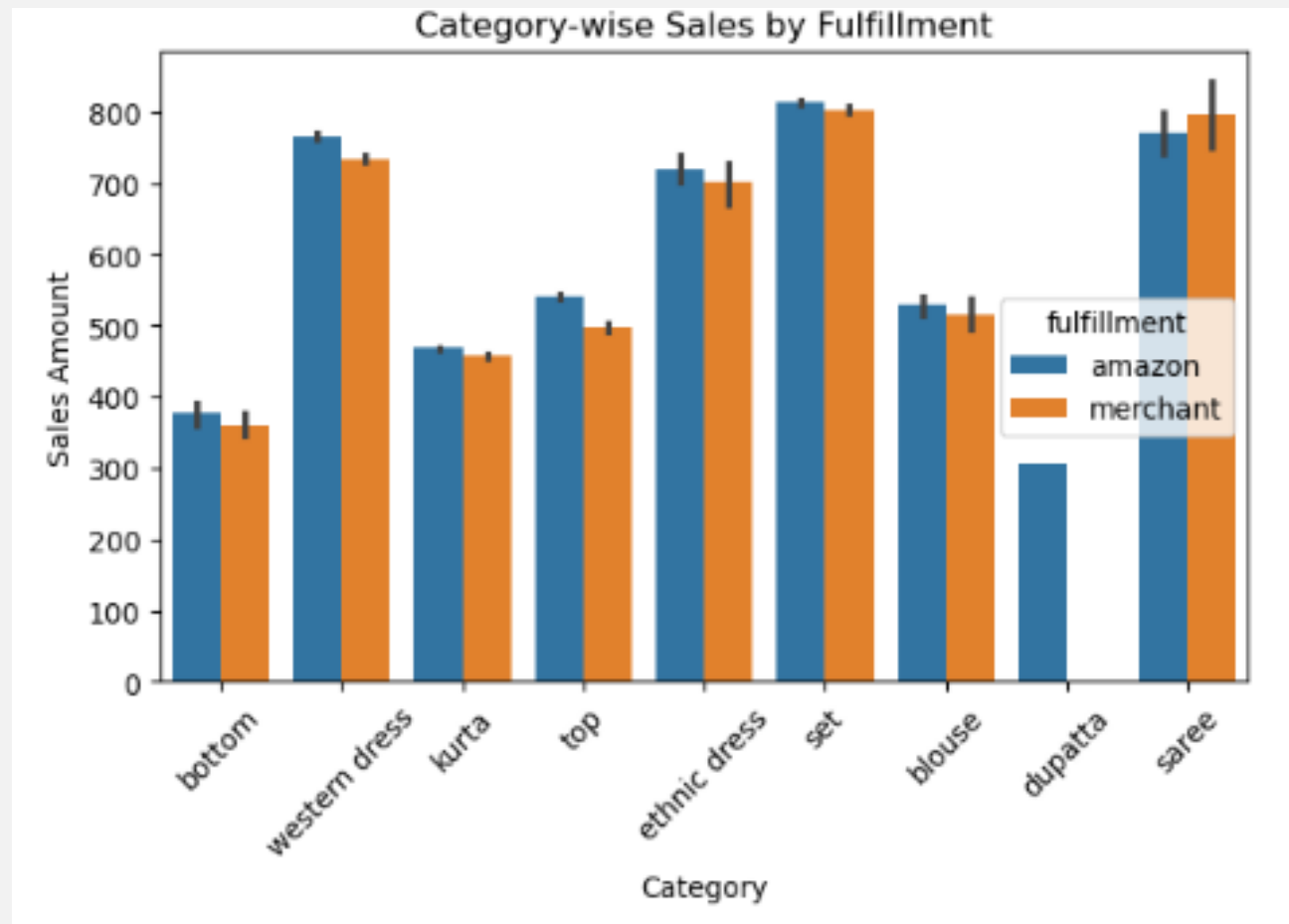
The correlation matrix looks at the connections between sales volume and encoded categorical data. The findings demonstrate a substantial positive relationship between shipping service level and fulfillment technique, suggesting a close relationship between these two operational elements. Additionally, there is a moderate association between courier status and the level of fulfillment and shipping services. All of the encoded variables, however, show very little association with the sales amount, indicating that characteristics like courier status, fulfillment type, shipping service level, and shipping location have little direct impact on the sales value. This suggests that factors relating to the product have a greater influence on sales volume than logistical or regional characteristics.

Visualization 15: Correlation matrix



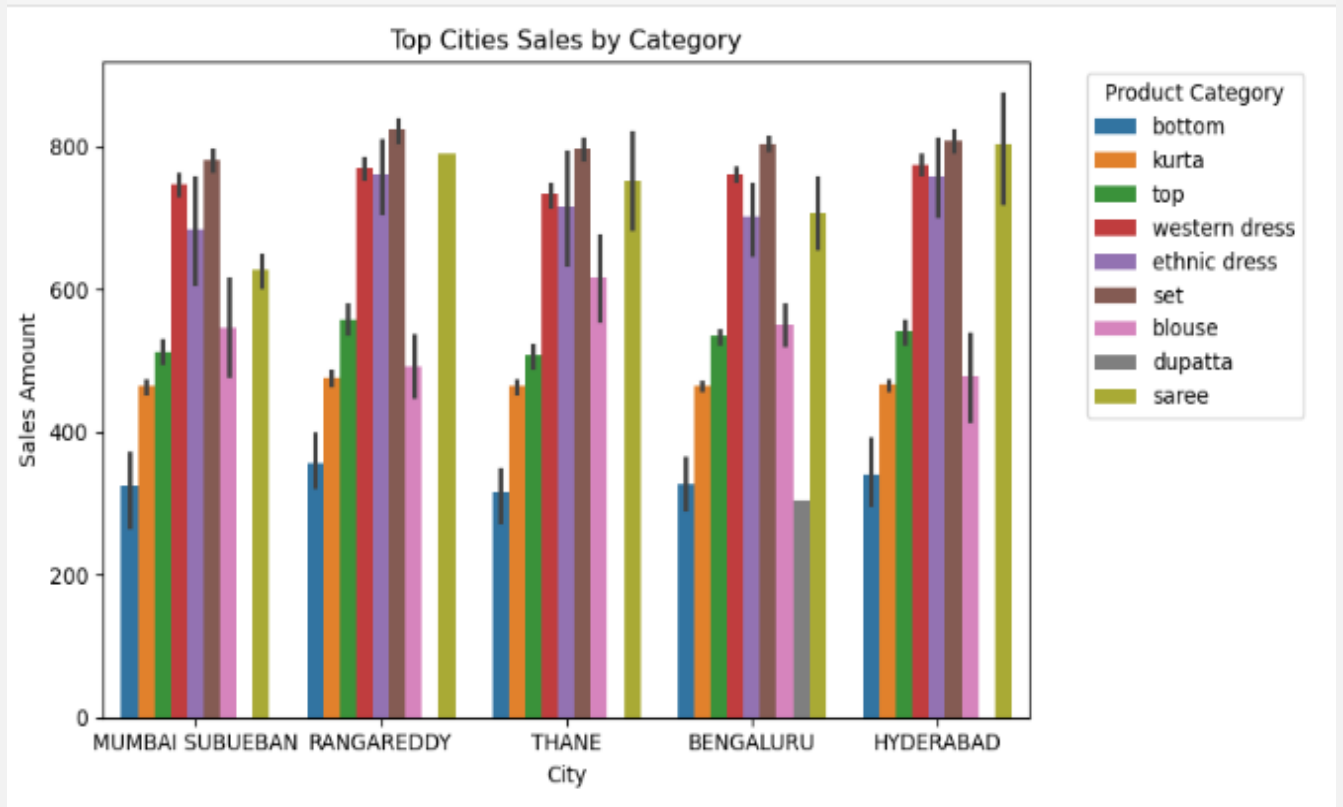
The correlation heatmap illustrates how strongly the operational, logistical, and sales variables that have been encoded are related to one another. The heatmap shows a strong positive association between shipping service level and fulfillment technique, suggesting that these two criteria are operationally closely related. Additionally, there is a moderate association between courier status and the level of fulfillment and shipping services. Conversely, there is very little link between the sales amount and the fulfillment type, courier status, shipping location, and shipping service level, indicating that these parameters have no direct impact on sales value. This suggests that logistical and regional factors have little bearing on sales performance.

Visualization 16: Category wise sales by fulfillment



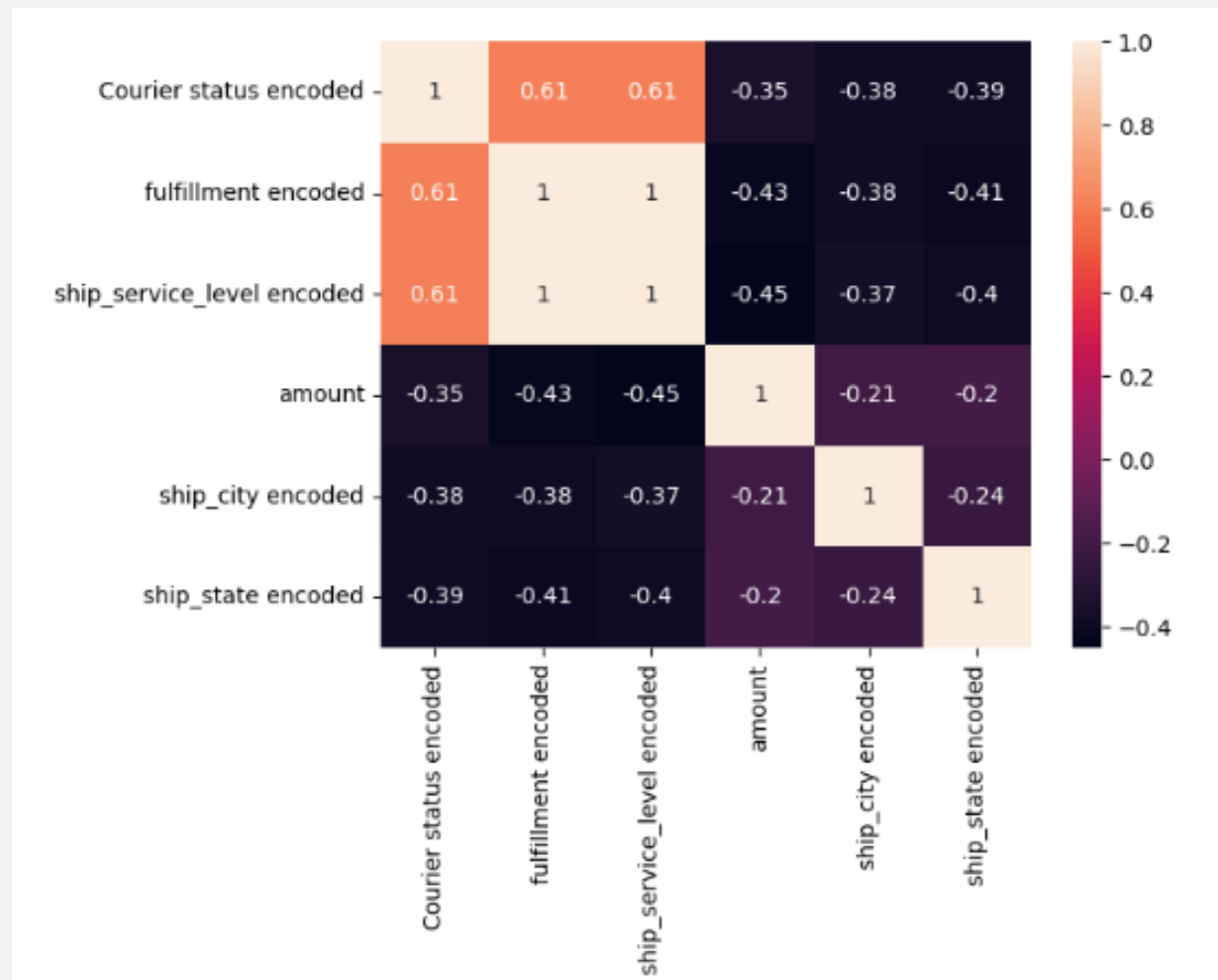
Sales figures for several product categories are compared in the bar chart according to the fulfillment type. The findings indicate that there are only slight differences in the sales values of merchant-fulfilled and Amazon-fulfilled orders across the majority of categories. Sales of sets, ethnic dresses, and sarees are comparatively greater under both fulfillment methods, however sales of bottoms and dupattas are lower. Overall, the visualization indicates that product category has a greater influence on sales quantity while fulfillment method has no discernible effect on category-wise sales performance.

Visualization 17: Top Cities sales according to category



Based on overall sales, the bar chart shows the sales distribution for the top five cities across various product categories. According to the graphic, major contributors to total revenue are Bengaluru, Hyderabad, Thane, Rangareddy, and Mumbai Suburban. Strong urban demand for these products is indicated by the continuously greater sales of sets, sarees, and ethnic dresses across these cities as compared to other categories. Similar category preferences across top-performing cities are seen in this data, which also shows important metropolitan areas as significant revenue drivers.

Visualization 18: Correlation



The associations between the variables of encoded courier status, fulfillment type, shipping service level, sales amount, and location are depicted in the correlation heatmap. Strong positive connections between courier status, fulfillment, and shipping service level are seen in the results, suggesting that logistics activities are tightly related. Conversely, there is a moderately negative association between sales quantity and certain operational factors, indicating that certain logistics configurations are not substantially linked to larger sales values. Furthermore, there are modest negative correlations between sales amount and state and city variables, suggesting that geographic location has little bearing on sales value. All things considered, the investigation confirms that operational aspects are interconnected, although other factors, including product attributes, significantly influence sales volume.

Overall interpretation:

Important insights into sales performance across product categories, geographies, fulfillment methods, and operational aspects can be gained from the comprehensive examination of the Amazon sales dataset. The results demonstrate that mid-range product sizes like M, L, and XL are in the highest demand, and that sales are heavily focused in particular product categories, especially sets, kurtas, and ethnic wear. At the regional level, major revenue providers are metropolitan states and cities such as Bengaluru, Hyderabad, Telangana, Maharashtra, and Karnataka. Additionally, the data shows that shipping service level, courier status, and fulfillment method have no direct effect on sales volume but are operationally significantly associated. Furthermore, correlation and distribution analysis indicate that product attributes and pricing, rather than logistics or geographic location, are what drive sales value. Overall, the study shows that concentrating on popular sizes, high-performing categories, and important urban areas can greatly improve corporate decision-making and sales success.

Business Recommendations:

Since sets, kurtas, ethnic dresses, and sarees account for the majority of total sales, it is advised that the company give priority to these high-performing product categories. Maintaining sufficient supply for medium, large, and extra-large sizes—which exhibit the highest client demand—should be the major goal of inventory planning. Targeted marketing campaigns and quicker delivery alternatives should be prioritized in high-revenue markets, such as Bengaluru, Hyderabad, Telangana, Maharashtra, Karnataka, and Mumbai Suburban, as these areas produce the biggest sales.

The data also shows that courier status and fulfillment method have little effect on sales value; consequently, rather than over-optimizing logistics setups, efforts should be focused on enhancing product variety, pricing tactics, and quality. To maximize inventory costs and overall profitability, low-performing sizes and categories should also be examined for possible redesign, discontinuance, or promotion.

Conclusion:

The thorough examination of the Amazon sales data reveals important trends in consumer buying patterns, product performance, and local demand. According to the report, high demand is concentrated in major urban areas, and a small number of product categories and sizes account for a significant portion of total sales. Despite their tight relationship, operational criteria including shipment service level, courier status, and fulfillment method have little effect on sales volume. This suggests that market demand and product-related factors, rather than logistical arrangements, are the main drivers of sales performance. All things considered, the research offers insightful information that can help with data-driven decision-making in marketing strategy, inventory management, and company expansion.